ContinuityGuard

Theme 1: Real-Time AI for Critical Decisions / System Anomaly Detection

Warn before disruption. Inspect the evidence. Preview the next step.

Native LinuxONE AI + accountable operator decisions

ANBU SELVAN T / SAHANA G / DHARSHINI E / POOJA V

Hindustan Institute of Technology and Science

Public code: github.com/ANBU-SELVAN-1907/RASCALS-IBM-DATATHON

Problem statement

Identify abnormal system activity as it happens and alert organizations before a major disruption. Target users: hospital IT operators.

1,294+

outpatient appointments postponed

NHS England reported this disruption across two affected trusts for 10-16 June 2024 after the Synnovis cyber incident.

Source: NHS England, 20 June 2024

Context only: our prototype was not tested on that incident and does not claim to detect or prevent ransomware.

The operational problem

Pool exhaustion, retry cascades and resource pressure can spread through shared dependencies before operators know where to investigate.

The decision we support

Which dependency needs attention, which software services matter most, and what should an operator inspect next?

Solution: a warning-to-decision pipeline

The browser uses TLS or SSH. Telemetry, inference, local generation and the journal stay in the LinuxONE VM.

JUPYTER CONTAINER / NATIVE s390x

Fit 5 candidates > separate calibration > freeze policy > fresh final test > hashed model

Ingest

Validated queue

Observe

Causal features

Score

Risk + repeat triage

Decide

Evidence + runbook

ISOLATION FOREST

Separate novelty score; not a probability or independent page trigger.

LOCAL GRANITE

Verified evidence wording; complete IDs and labeled fallback.

RESPONSE PREVIEW

Same-workload action/no-action branches in the simulator only.

A small model, selected for the actual task

Predict a new sustained service-objective breach within 60 event-time seconds using current and past observations.

240
Train runs

60
Calibration

120
Validation

180
Fresh test runs

Calibrated logistic risk

Scaled logistic regression won validation. Initial warnings stay intact; learned risk or observed breach permits repeats. Independent forecast paging is disabled.

Evidence of separation

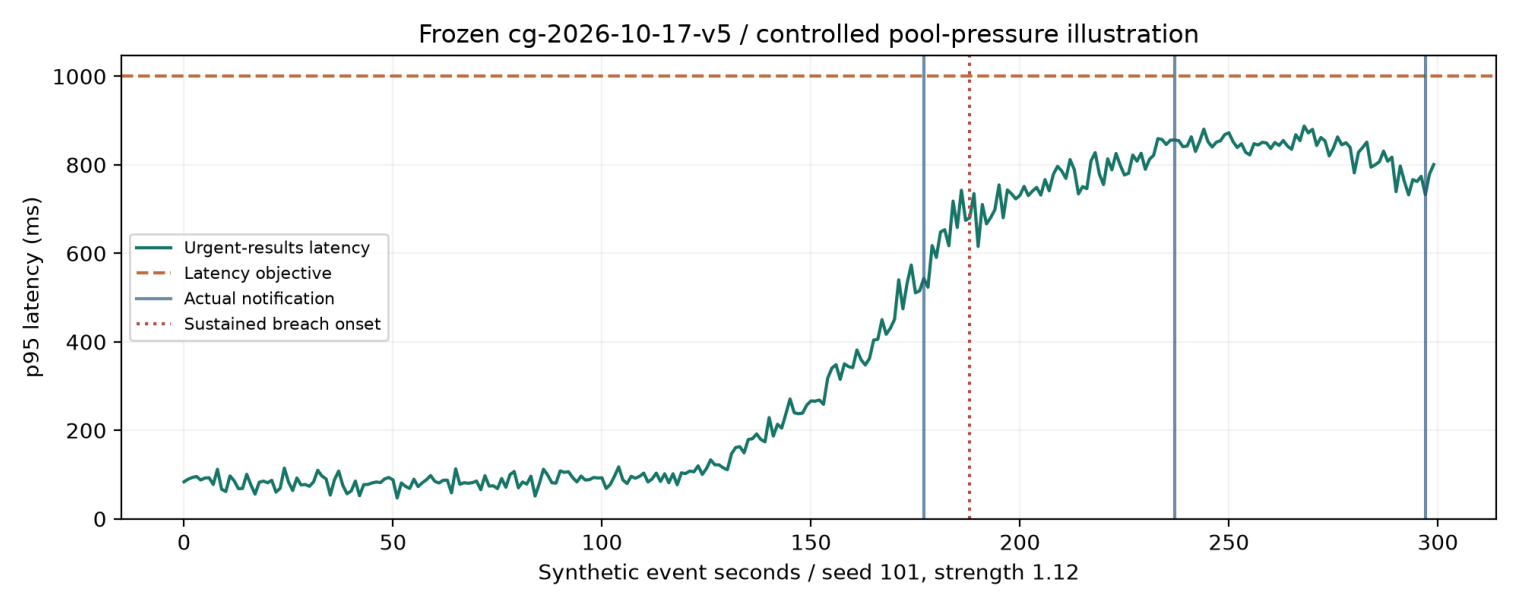
26 causal features; whole runs and disjoint seeds. No scenario, onset, future values or fault labels enter the predictors.

Forecast PR-AUC 0.818 / prevalence 4.6% / Brier 0.01691

V5: matched 60-second horizon; shared alert scheduler; earlier V1-V4 results retained.

From incoming telemetry to retained warning evidence

Actual frozen-model scenario output. The deployed dashboard processes the same controlled synthetic service telemetry.



Retained warning evidence

Waiting queue

22.2 requests

Connection pool occupancy

90.1 %

Latency p95

696.8 ms

Actual selected incident; measurements are evidence, not causal attribution.

The breach marker also includes error fraction above 10%; the latency curve alone need not cross 1,000 ms.

Streaming services > persistent warning > measurements > declared dependency and runbook

Higher precision with retained early detection

Frozen V5 test: 180 synthetic runs; correlated services within a run. No production hospital validation.

Detector	Early recall	Precision	Median lead*	Late	Missed
Static thresholds	96.3%	43.5%	11.0 s	4	1
Robust statistics	98.5%	11.2%	26.0 s	2	0
Isolation Forest	99.3%	14.3%	35.5 s	0	1
ContinuityGuard	**96.3%**	**45.3%**	**11.0 s**	**4**	**1**

Early warnings: 130/135 vs 130/135. Precision: 45.3% vs 43.5%. Unmatched notifications: 162 vs 174. Lead and misses are equal in this test.

Paired 95% interval for precision gain: 0.63 to 3.15 percentage points. 500 whole-run resamples; simulator-conditional.

*Among early detections. Zero healthy alerts in 30 service-hours does not prove zero real false alarms.

The response is inspectable and bounded

Evidence, service priorities, approved investigation steps and a reproducible response preview form one decision workflow.

ACTUAL SAVED APPLICATION OUTPUT

database needs operator investigation. Investigate database connection pressure. Supporting measurements: Waiting queue 22.2 requests (E3); Connection pool occupancy 90.1 % (E1); Latency p95 696.8 ms (E5).

+309 successful urgent-results transactions

Modeled breach: 104 s without action to 0 s with action. Tradeoff: 1175 optional requests deferred.

Source: evidence/demo-verification.json / synthetic scenario

Predictive + generative, locally verified

Granite generated schema-validated wording on s390x in 18.75 seconds in the Jupyter proof. Every evidence ID cited in the summary is retained.

The preview compares allowlisted actions under identical future demand/noise. It reports benefits with deferred optional work. It executes no real remediation.

Suspected source is a hypothesis, not causal proof.

Platform use that judges can verify

Native AI execution, actual ingestion and container controls are exposed as evidence.

43

native tests passed

Checks cover leakage, online/offline parity, time gaps, invalid inputs, provenance, overload, consumer failure, model corruption and grounded fallback.

1000/1000 events processed

Real local HTTP path: 100 offered events/s for 10 s. Queue-ack p95 7.982 ms; queue wait and inference are distinct boundaries.

Application containment

UID 1000 / read-only root / capabilities dropped / no-new-privileges / 2 GiB memory / 1.5 CPU quota / Authenticated TLS / SSH

ACTUAL DEPLOYED API PROOF

Architecture: s390x / process UID: 1000

Read-only root: True

Model: cg-2026-10-17-v5

SHA-256: ca4328b8ed2099ea4a007e75e626a4a6...

Saved API values; full model hash and container verification in evidence/. No accelerator claim.

Scenarios: normal change is not always failure

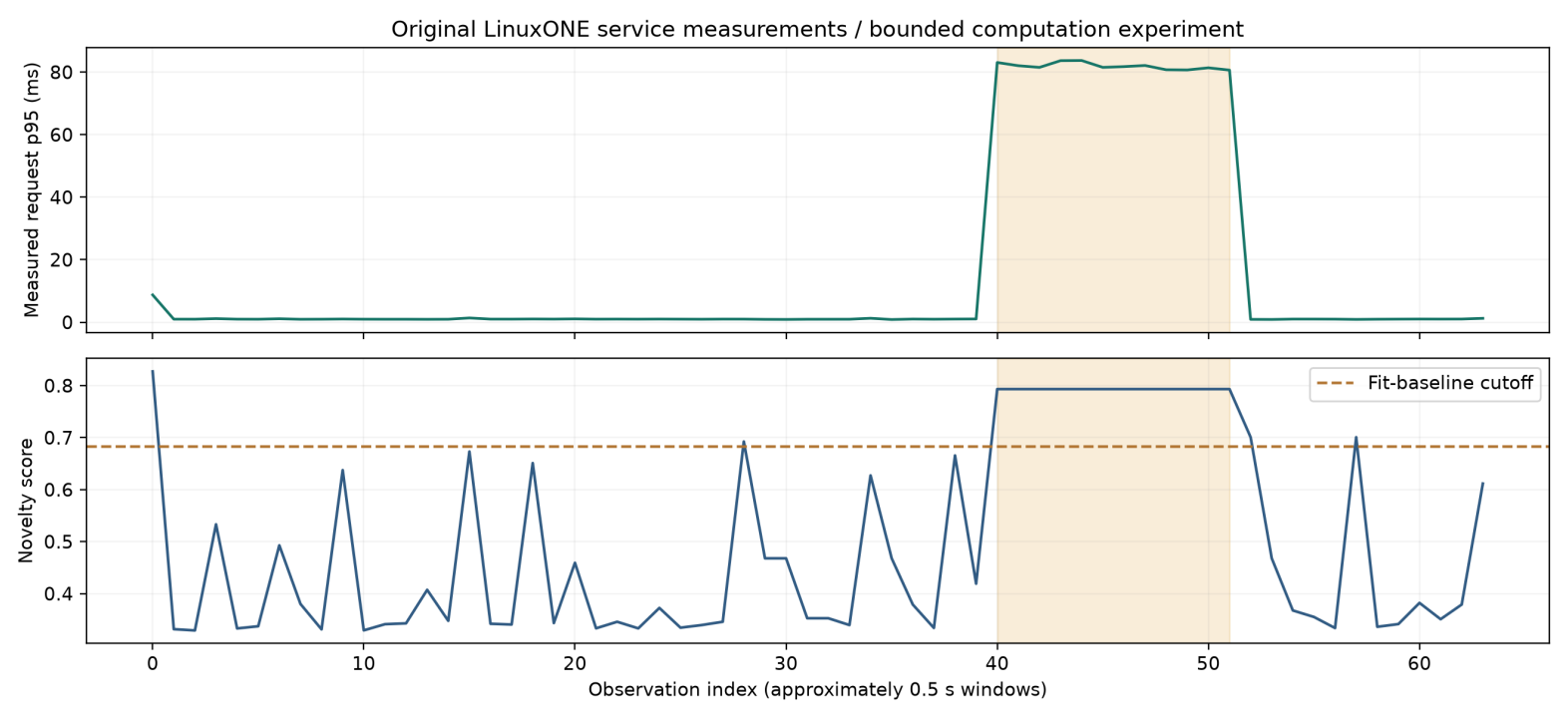
Fixed seed 101 / severity 1.12 / frozen V5 model. These are synthetic illustrations, not a new independent test.

Scenario	Notifications	First urgent alert	Urgent breach
healthy	0	None	None
healthy surge	0	None	None
healthy shift	0	None	None
pool exhaustion	12	177 s	188 s
retry storm	9	196 s	219 s
memory pressure	1	None	None
abrupt failure	4	94 s	94 s

Counts include episode rechecks, not unique outages. Pool/retry examples warn before breach. The abrupt example is detected at onset, without an early-warning claim.

Original data: measured on this LinuxONE VM

64 actual service observations / bounded request computation / raw CSV, timestamps and SHA-256 retained. No patient or hospital data.



Observed baseline check

0.98 ms

Controlled compute pressure

81.64 ms

Median measured request p95 across windows. All 12 pressure windows exceeded the fit-baseline novelty cutoff; 2 recovery windows also did.

Separate small novelty experiment. It does not validate the simulator-trained breach model, production recall or real outage prevention.

What makes the integration distinctive

The useful output is an inspectable operational decision, not an unexplained anomaly score.

01

Retain the warning evidence

Measurements and declared downstream priorities remain attached to each incident.

02

Constrain local explanation

Granite selects verified wording and complete evidence IDs; invalid generation is explicit.

03

Preview the tradeoff

Compare allowlisted action and no action under the same synthetic future workload.

Established algorithms; original project integration. No world-first or autonomous-prevention claim.

Learnings during the datathon

Changes we can explain from actual implementation, tests and corrected evidence.

01 Native model selection

A smaller calibrated model won the notification objective; model size alone did not decide it.

02 Alert lifecycles

Old alerts need bounded rechecks. Preserve first warnings before filtering repeats.

03 Fair evaluation

Freeze before fresh tests, apply the same scheduler, and retain rejected results.

04 Grounded generation

Every evidence reference must match generated wording; fallback must be labeled.

05 Data provenance

Measured service behavior, controlled simulation and hospital data are different evidence.

Implementation and IBM Toolkit fit

Six native Jupyter notebooks, a deployed service, and a documented optional serving path.

01 / Prepare

Provision the VM, complete assigned labs and retain persistent Jupyter storage.

02 / Train and verify

Fit/calibrate/select, then review fresh final results and engineering tests.

03 / Measure and deploy

Collect original service measurements; build the resource-limited app and verify its journal.

04 / Demonstrate

Open authenticated TLS by IP, inspect an alert and explain/preview the next step.

IBM AI Toolkit

Applied deployment guidance: separate functions, non-root app, read-only root, no-new-privileges, TLS and authentication.

Snap ML preparation: a separate supported-family Random Forest candidate was trained and ONNX-checked on s390x. It is not the deployed model.

Registry access is unavailable. Supported Toolkit container execution and AI acceleration are not claimed.

An accountable path from warning to action

Original integration and measured implementation, with limits that operators and judges can inspect.

Community value

Earlier investigation of shared service failures; preserve attention for essential software services.

Engineering evidence

Native notebooks, actual API ingestion, model hashes, failure tests and local explanations.

Next validation

Authorized real telemetry, operator feedback, domain calibration and reviewed runbooks before operational use.

github.com/ANBU-SELVAN-1907/RASCALS-IBM-DATATHON

Both sample labs executed before project work; confirm organizer mapping to the required categories. Review/upload the <=3 minute video and submit the portal manually.

AI assistance is disclosed. Open-source resources and a criterion-by-criterion evidence map are in the repository. No score or award is guaranteed.